How to install Linux on a Macintosh and dual boot with macOS

Got one of those shiny Mac laptops, but Linux has you realising computer freedom is best?

This is the definitive guide!

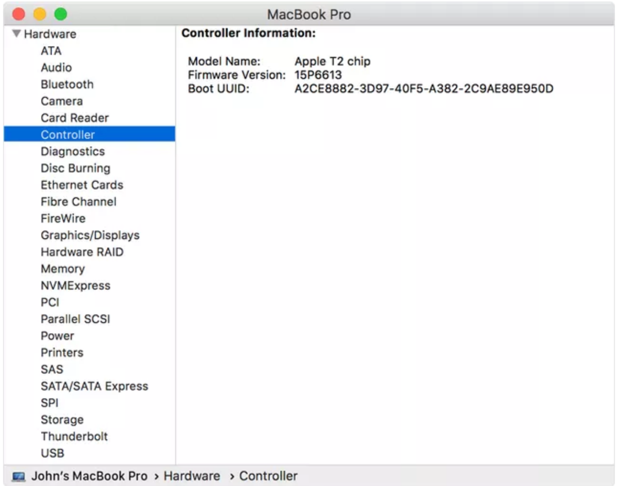
UPDATED FEBRUARY 2019

Using a Macintosh is (mainly) a delight. The hardware is solid, fast, and beautiful, but over time, macOS has become dumbed down and in some places, downright silly. I long since realised that I could do exactly what I wanted to do with my macbook using Linux, rather than being encumbered by having to follow the 'Apple' way of doing things. I never looked back. Here's the definitive guide to installing Linux on a Mac.

DISCLAIMER: This is an advanced tutorial which sometimes works at the command line and can cause irreparable damage to your data. If you do proceed, make sure you have backed everything up with TimeMachine or such like tools. The Ultimate Linux Newbie Guide cannot be held responsible for any damage caused as a result of following this tutorial.

This tutorial has been tested on a late 2013 Macbook Pro Retina 15", however it should work with any EFI based Mac (more on that in a bit). The EFI based Macintosh started around 2008 (you can check the list of the Apple EFI systems **here**). This should include Macbook Pros, Macbook Air, iMac and probably Mac Pro's...

Update: Apple's new P2 'Secure boot' chip



To find out if you have the T2 chip:
1. press and hold the Option key while choosing Apple () menu > System Information.
2. In the sidebar, select either Controller or iBridge, depending on the version of macOS in use.
3. If you see "Apple T2 chip" on the right, your Mac has the Apple T2 Security Chip..

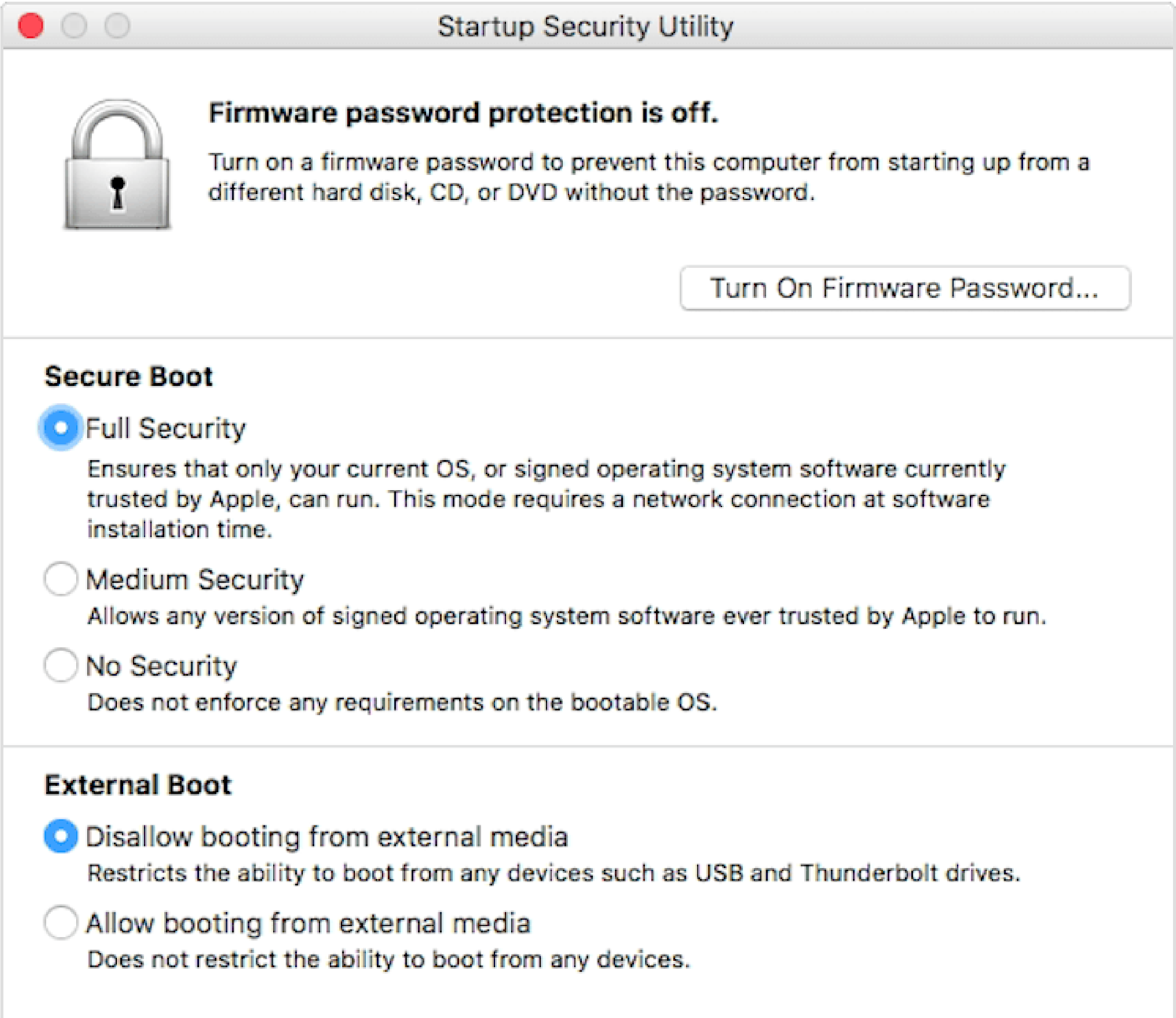
Unfortunately, from 2018, Apple decided to add a new 'secure boot' T2 chip into their mac hardware. This basically means that you can't use anything other than macOS on Apple hardware, however it is possible to switch off secure boot. See the image above to show you how to find out if your machine has the T2 chip.

If you have said T2 chip, then you may be completely out of luck for Linux on your mac, or at least for now. If you want, you can try to disable the secure boot option in order to install Linux on your mac. Note that I haven't tested this (I don't have a new mac), so please let me know if it works for you.

Disabling the Secure Boot Option

You'll need to start your mac into the Recovery mode and launch the Startup Security Utility. To do this, just follow these steps:

1. Turn on your Mac (or restart it if it's already on), then press and hold Command (⌘)-R immediately after you see the Apple logo. Your Mac starts up from macOS Recovery.
2. When you see the macOS Utilities window, choose Utilities > Startup Security Utility from the menu bar.
3. When you're asked to authenticate, click Enter macOS Password, then choose an administrator account and enter its password.
4. Now look at the options, there should be an option for 'Secure Boot'. Switch it off by selecting 'No security'.
5. There should also be an option about 'External Boot'. Ensure that this is set to 'Allow booting from external media'.



The Startup Security Utility defaults enforce the highest security by default. This won't let you install Linux on your mac, let alone boot from a USB stick.

Dual Booting with Mac OS (yes, you can keep MacOS!)

I am writing this assuming that you want to keep MacOS on your hard drive and that you wish to dual-boot it at any time. You should have plenty of free space on your disk drive (the more the better), so either delete some cruft or move some of your old data onto a separate external archive hard drive (because I know you got one or ten of them lying around!).

I used MacOS Mojave, which is the latest version of macOS at the time of writing. Recently Apple introduced a 'security feature' called 'SIP' (System Integrity Protection) which you will additionally have to overcome if you are using El Capitan or newer. More on that in a bit. We will be installing Ubuntu. This tutorial was written with Ubuntu , but this should apply to any Linux distro more or less, although your mileage may vary with Video stuff particularly.

NOTE: You may have to install an EFI boot manager (rEFInd) and/or do a few gnarly things to get your hardware working before it is Linux ready, so if you get stuck at any point, read towards the end part of this guide.

The tutorial you are about to read has four main steps. These are:

- Downloading and 'burning' your Linux distro of choice to a USB stick.
- Partitioning your hard drive
- Installing Linux
- Finishing up, which includes: Adding driver. Disabling SPI and enabling EFI. Nice to have items, including being able to see your Macintosh files from Linux.

Step 1: Downloading and 'burning' your Linux distro image of choice to a USB stick.

Next, unless you haven't already downloaded the Linux distribution of your choice, it's time to go grab it. You'll find that you'll download a .iso file, which we will need to 'burn' onto a USB stick. Make sure you have a 4GB or bigger USB stick that you don't care about deleting ready for use.

For this particular tutorial, we are using Ubuntu, however most other Linux distributions should work. Using more hard-ass systems like Arch or Slackware, or even Debian, this will be more challenging. This guide is challenging enough, so do what you will, but I recommend you stick to the easier distros to begin with like Ubuntu or Linux Mint.

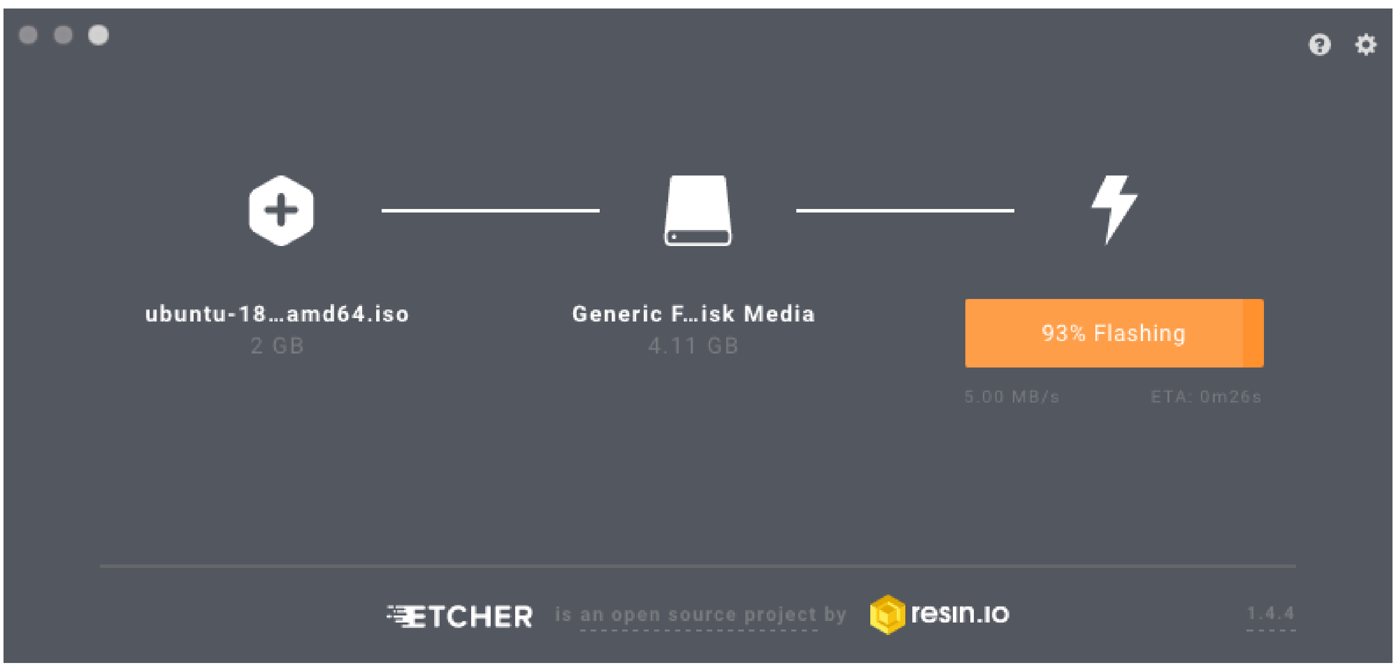
Make sure you download the x64 version of the distribution you choose, if there is an EFI boot version, choose that also.

Using Etcher to 'burn' your ISO image to a USB stick.

There is now a snazzy tool called Etcher (you can download it for free from balena.io/etcher. This would now be my choice for downloading and burning a Linux distribution download to a USB stick because it's literally as easy as popping in your USB stick and pressing go!

Now that you've got your ISO file downloaded, and you've downloaded BalenaEtcher, Fire up Etcher, and follow these steps:

- Click 'Select Image'. Select the Linux ISO file that you just downloaded.
- Insert your USB stick that you want to put the Linux distribution onto (note it will be completely wiped).
- Click 'Select Drive'. In many cases, this might not even be necessary (Etcher is clever enough to see the USB stick and select it for you).
- Click Flash!



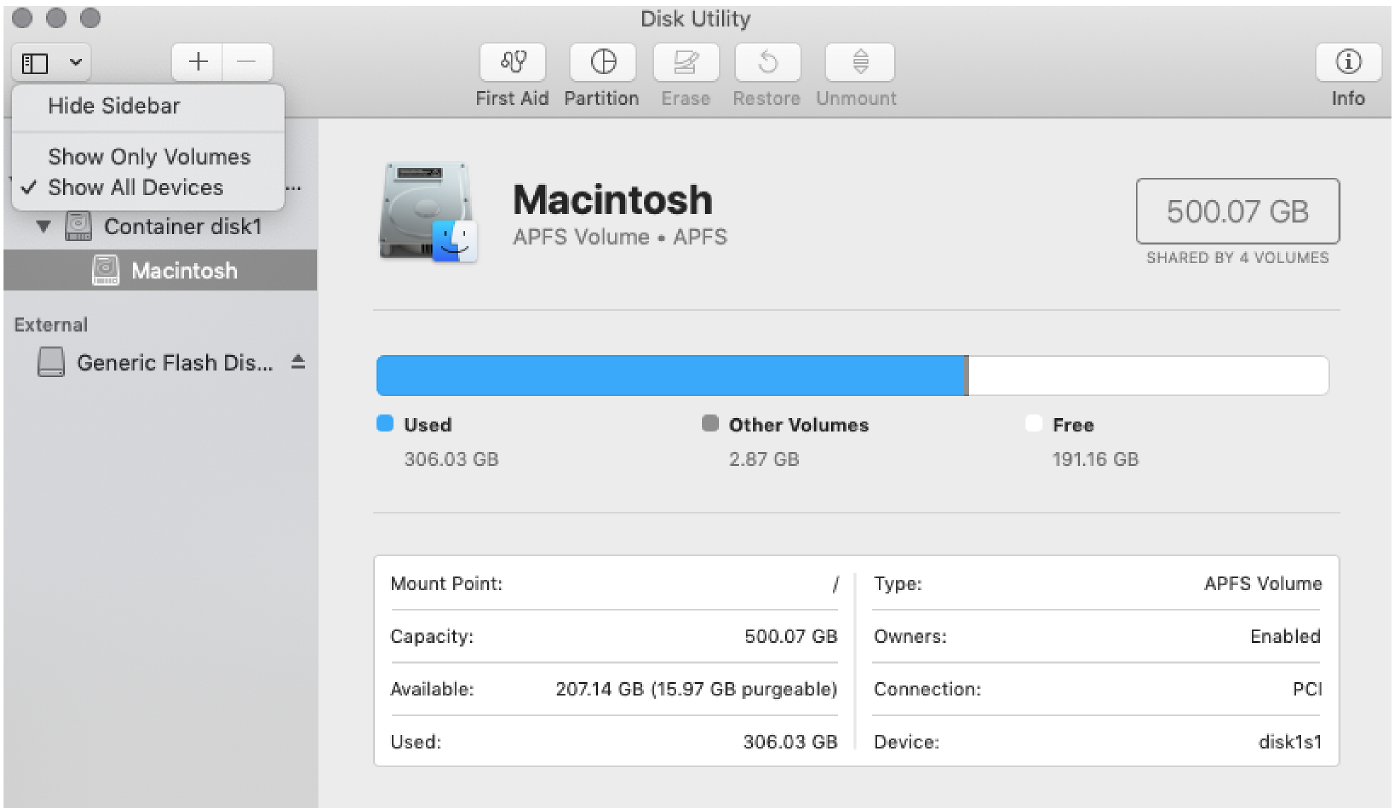
Etcher in action - a super quick and easy tool to put your Linux ISOs onto a USB stick.

Yep, that's it! If there is any reason why you can't get this to work, then you can follow the 'old fashioned' way of doing it over on this short guide.

Step 2: Partitioning your Macintosh hard drive

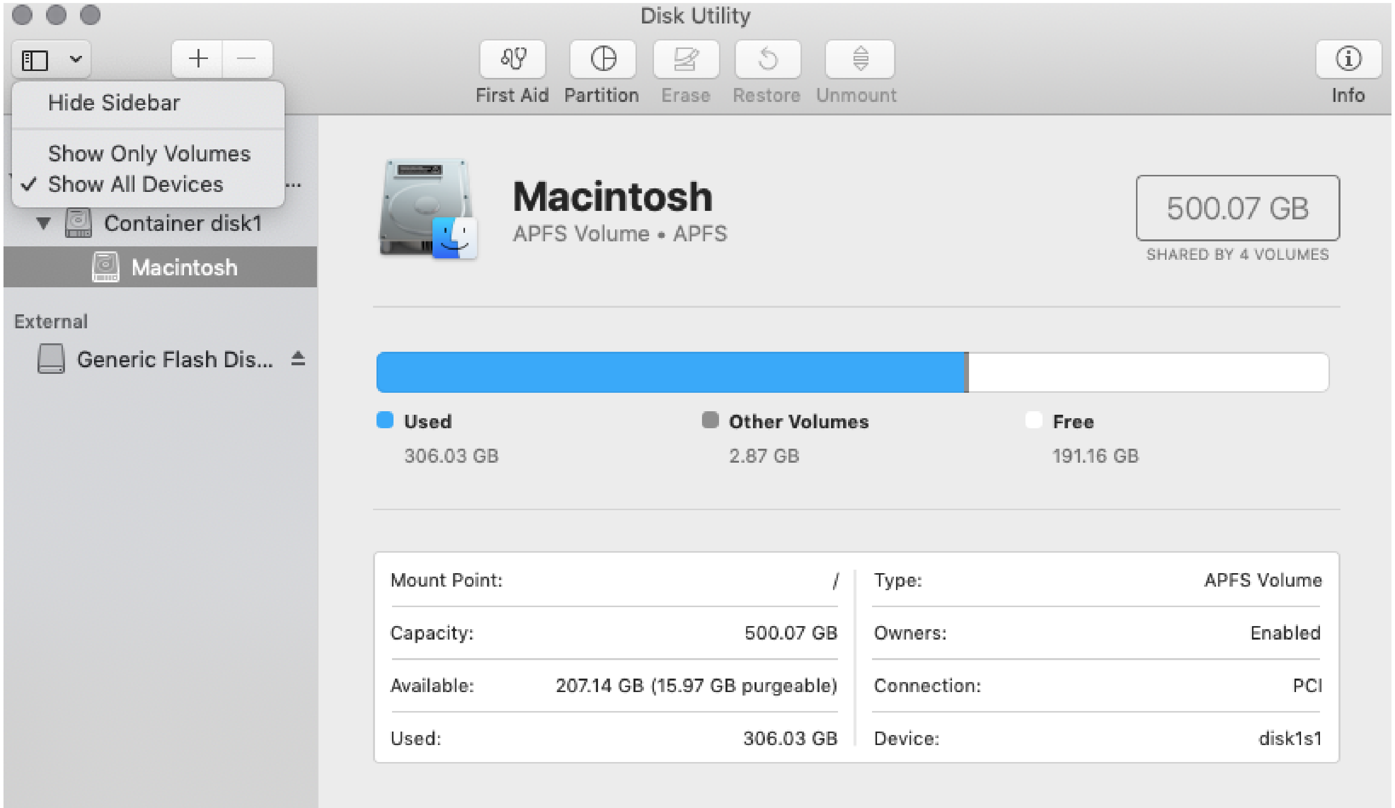
This step chops your disk up the way you want it - some space for macOS, some space for Linux. This is called 'Partitioning'. Make sure that you delete as much junk from your mac before you start, that way you can give as much space as you can to Linux.

To modify your partition table in macOS simply look in your **Utilities** folder, you'll find Apple's **Disk Utility**. If you like, quickly scan your hard drive for errors, just to make sure it's all sweet before we get down to business. Repair any errors you may find.



Once you are ready, you will see a list of internal drives on the left hand side. Your Disk Utility may look different if you are using an older version of macOS, but it still offers the ability to resize a volume.

If you are using a recent version of MacOS, you'll find that macOS now uses a notion of disk containers. To see everything that's going on, you'll need to click the icon to the top left, it should show you 'Show Only Volumes' or 'Show All devices'. Select Show All Devices. The screenshot below shows this action.

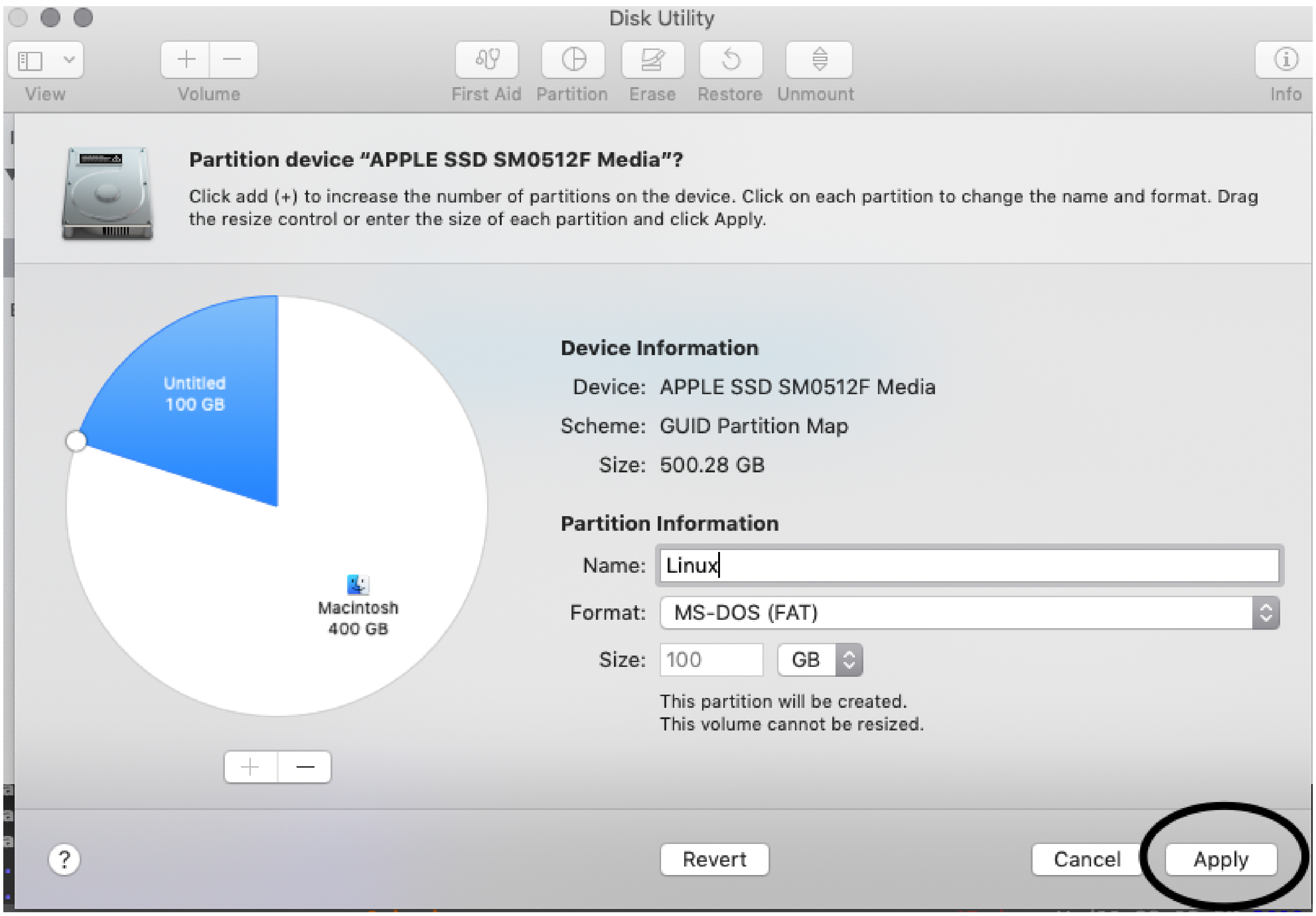


Select 'Show All Devices' from the top left menu in Disk Utility.

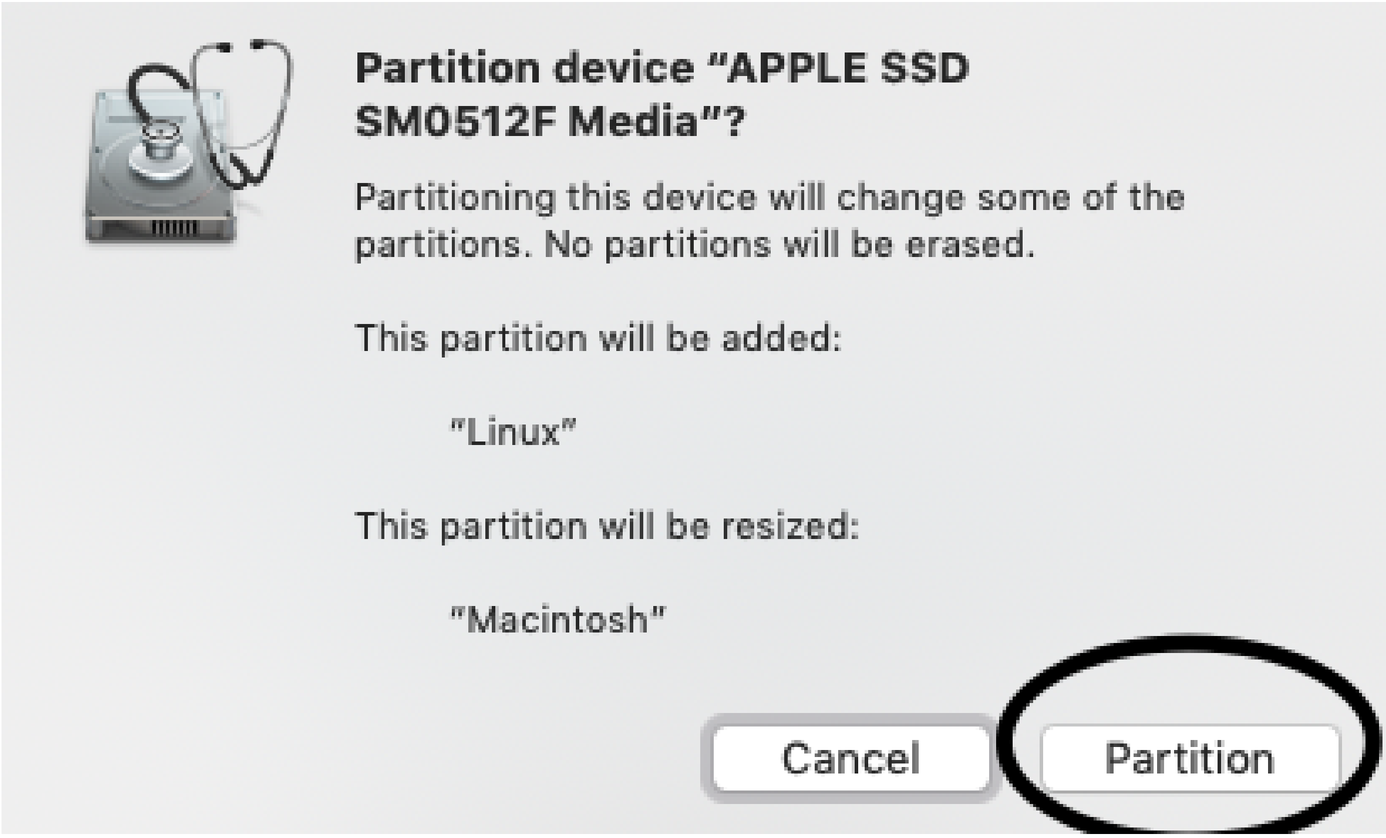
On the hard drive that your macOS partition exists on, click on the top drive, not any subsequent partitions listed below it. Click on the 'partition' button (it looks like a pie chart in modern versions of the utility).

In newer versions of macOS, they prefer you to use these 'container volumes'. That's fine for macOS, but you want a partition to put Linux on. If you see the above dialogue box appear, make sure you click 'Partition'.

Next, you'll see the partition pie chart. You will see you can move the slider around the pie to resize your partition(s). Pull the size slider back for the Mac OS partition to release the free space on the disk. Make a blank partition until you have enough space for your new Linux system. Make it as much space as you are willing to, I gave my Linux partition 100 GB.

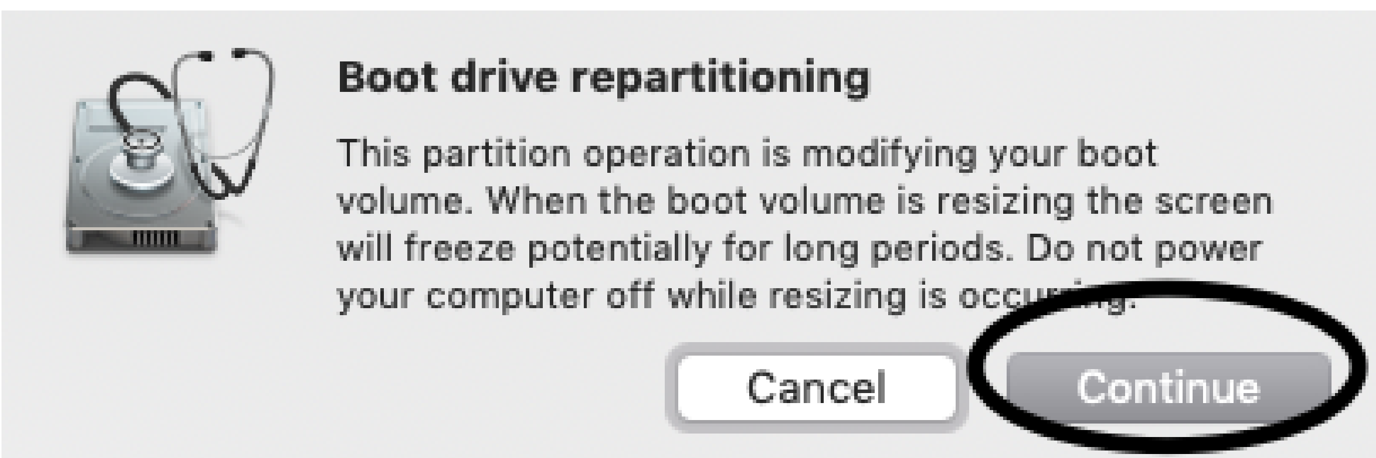


It's essential that you choose to format the partition as MS-DOS (FAT) format. I gave it the name 'Linux' so that it's easy to tell what it is. Once you've done that, click Apply.



Click the Partition button.

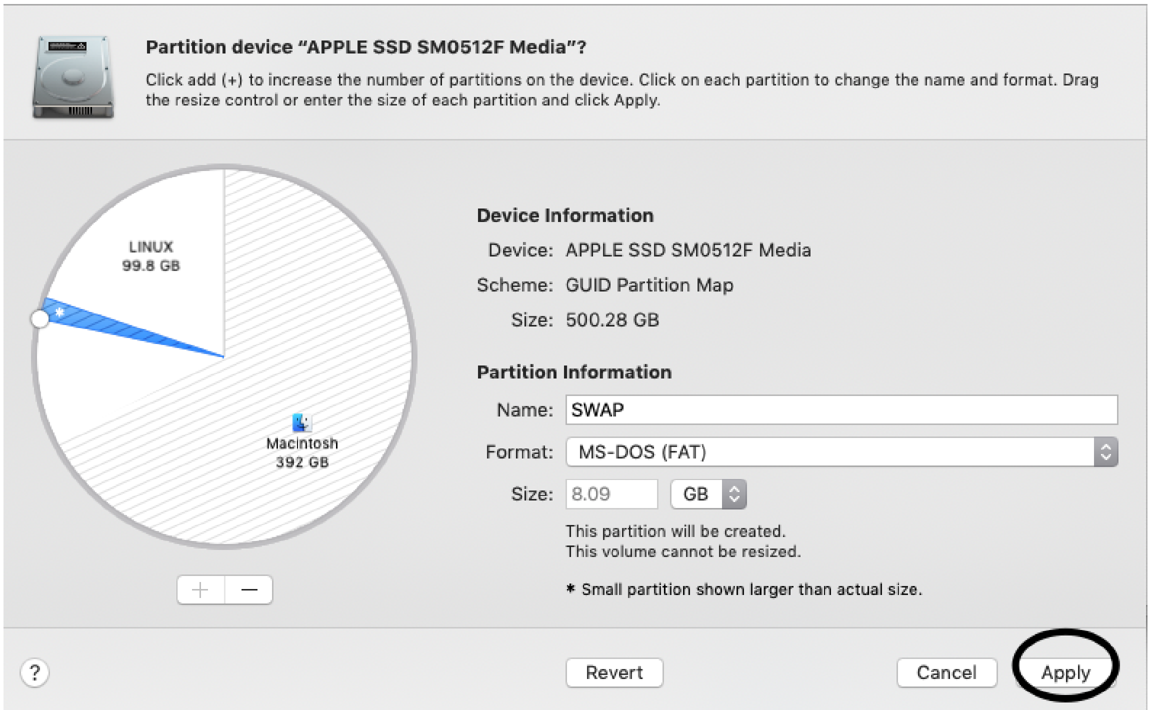
You'll see the box to the left. Apply the changes by clicking the Partition button and let the resize operation complete. If you have an SSD, this should be relatively quick (a few minutes). For older hard drives, this is going to take some time!

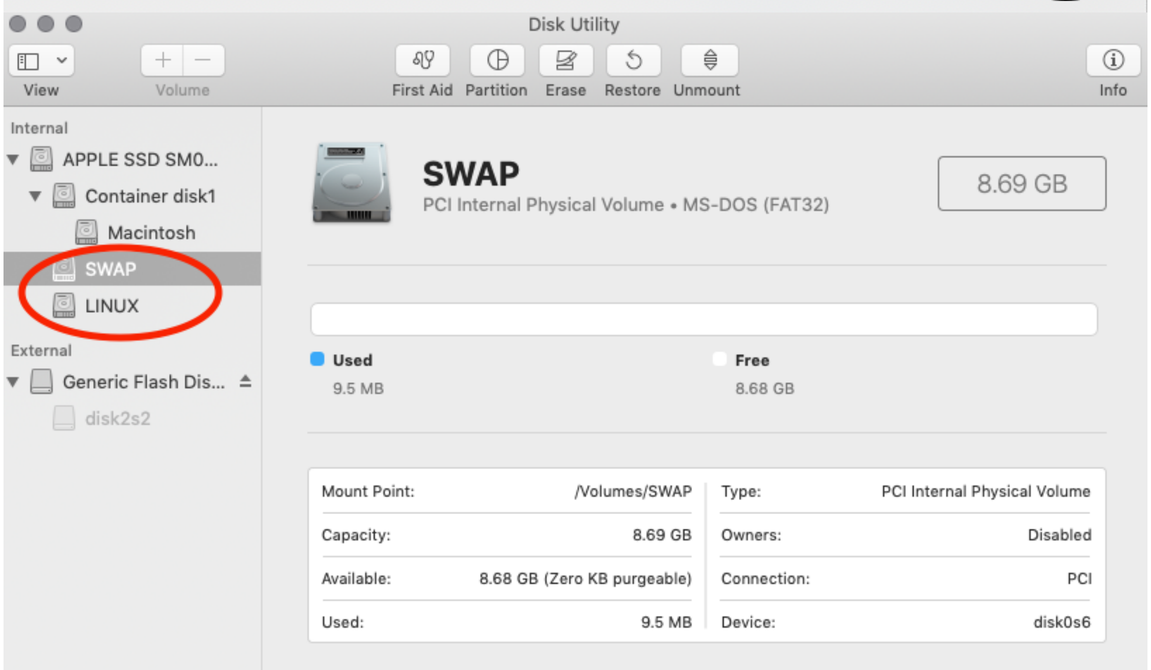


You'll probably see this message, just click 'Continue'.

NOTE: I also recommend also making a swap partition, although this isn't completely necessary. To do this, simply follow the steps you did above but make a smaller partition, eg 8GB.

The below screenshots show the creation of a SWAP partition and the final 'picture' of what your macOS disk should look like.

- 

- 

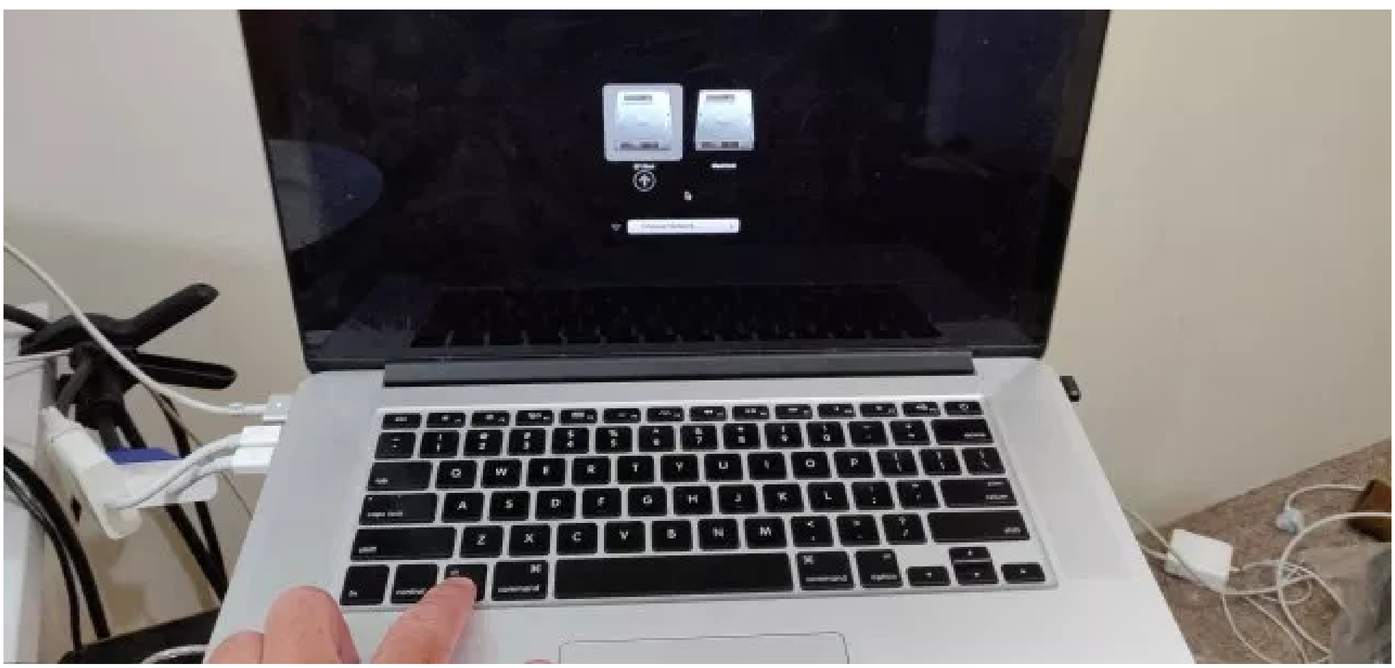
Step 3: Installing Linux on that Mac!

Woo-hoo! This is the fun part! Now we get to install the operating system that your Macintosh has been longing for.

Using a USB or Thunderbolt Ethernet Adapter is going to save you a lot of headaches!

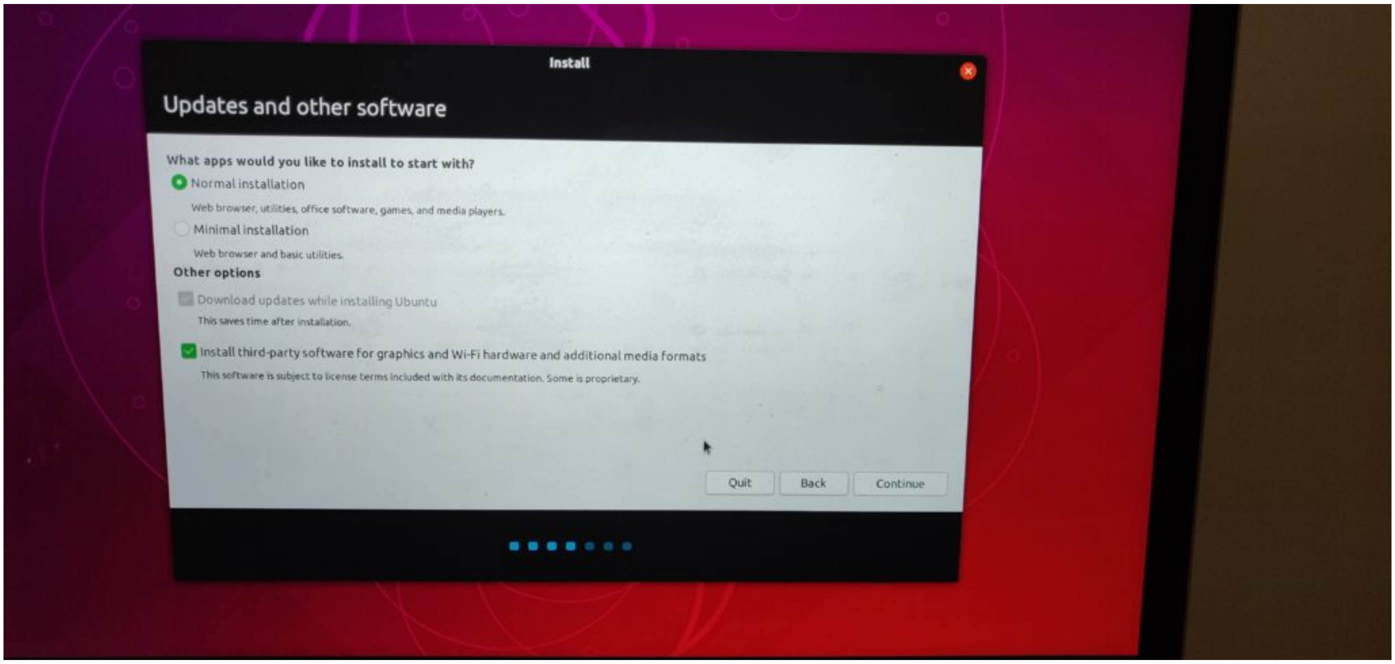
Switch your Macintosh off completely. Connect your Ethernet to Thunderbolt adapter (or USB Ethernet Adapter) and your USB drive we made earlier. If you don't have one of those ethernet adapters, life is going to be tricky for you, you are going to have to download the wireless drivers and install them manually to get things working. If you don't have one of the adapters, ask a friend for one, or buy one cheap from Ebay or such like. It will save your sanity.

Turn on your computer and hold down the option/alt key. You'll see a menu pop up which you can see your Macintosh HD as well as the USB stick. It will be named EFI Boot or something similar. Use the cursor keys or mouse to select that and hit return. PS: Make sure you revert to using your laptop's keyboard and mouse for the time being (your bluetooth keyboard, and probably your mouse won't work until paired).



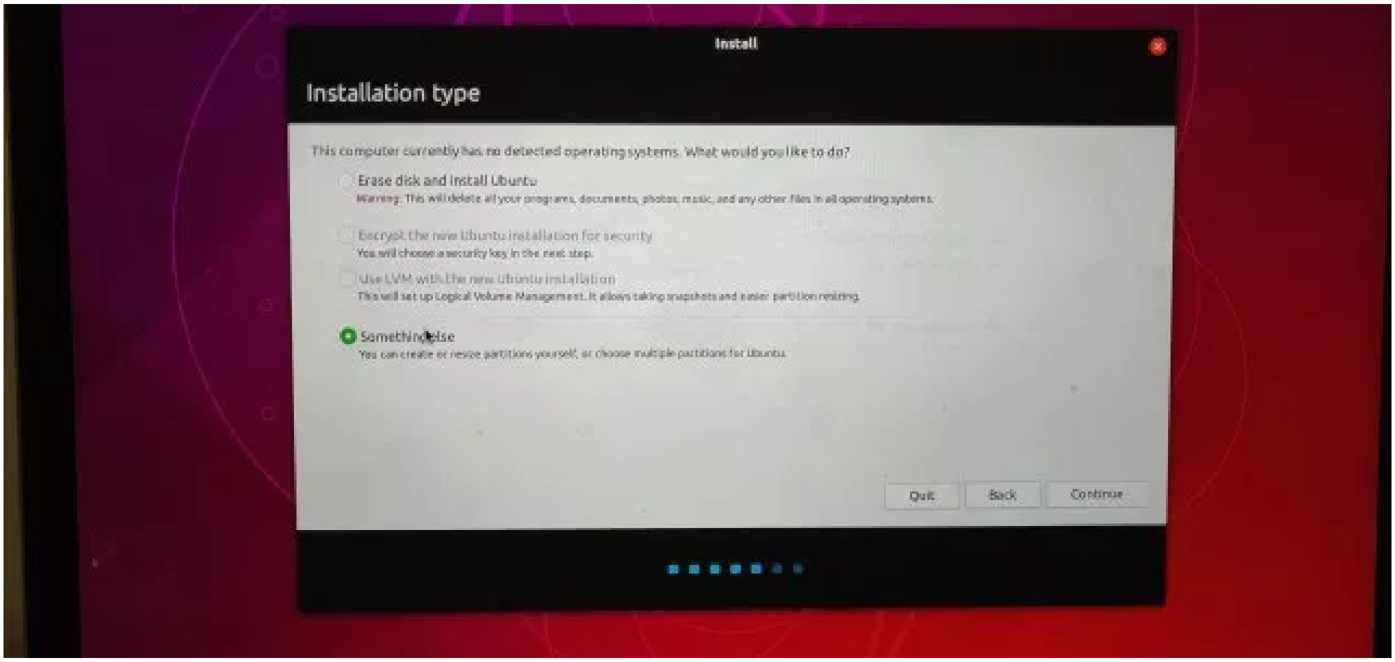
Hold down the alt/option key whilst starting up your mac and you'll see this screen.

Shortly after, you'll see the Ubuntu installer start up. Follow through the steps as usual. You'll get to a screen that says 'Updates and other software'. Make sure you tick the box that says Install third-party software for graphics and Wi-Fi.



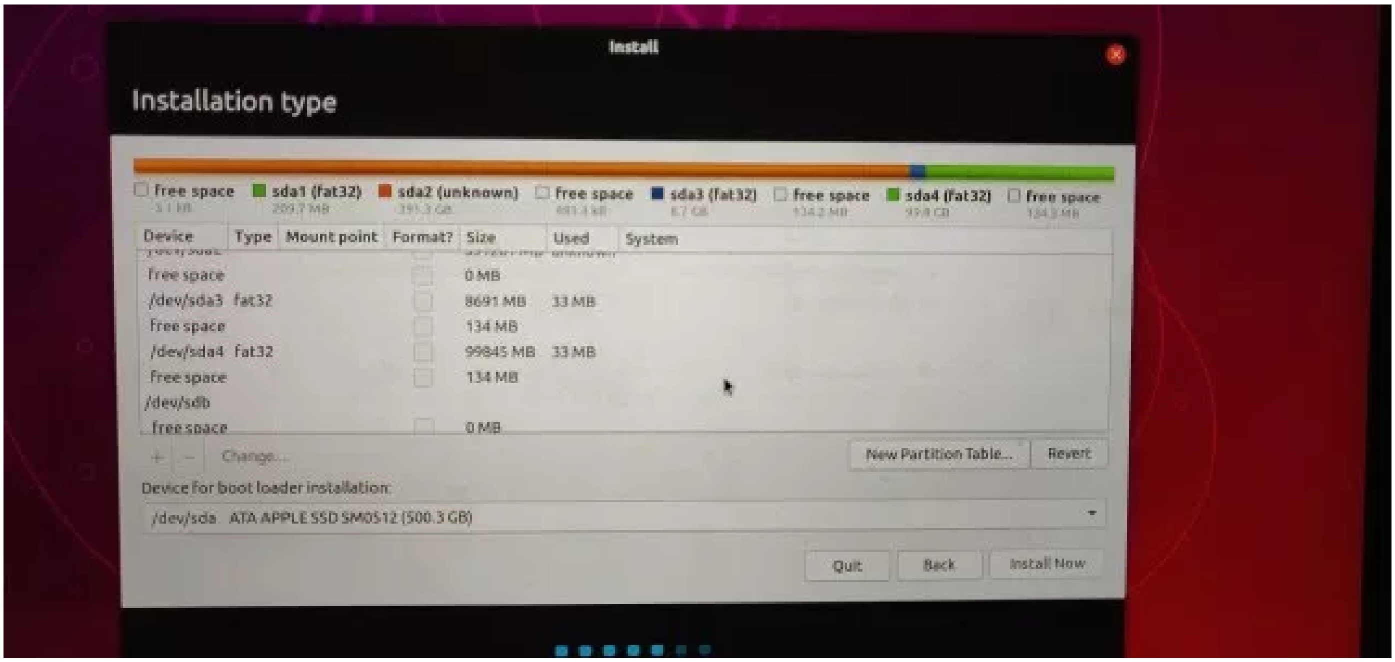
Make sure to select a normal installation, and tick the box 'Install third-party software'.

The next step, and arguably the most important step in the entire process is about installing Linux on the partitions you previously configured in the Disk Utility. You'll see a dialogue saying 'Installation type'. Make sure that you choose the option 'Something else'. If you select the other options, these will delete your installation of macOS and make you have a bad day (TM).



In installation type, ensure you choose 'Something else'.

In the next dialogue, you'll see the partition table (and probably some empty partitions too). If you created a swap partition as per my example, you'll see two FAT32 partitions. One will be the small 8GB SWAP partition, the other 100GB (in my case) is the main Linux partition.

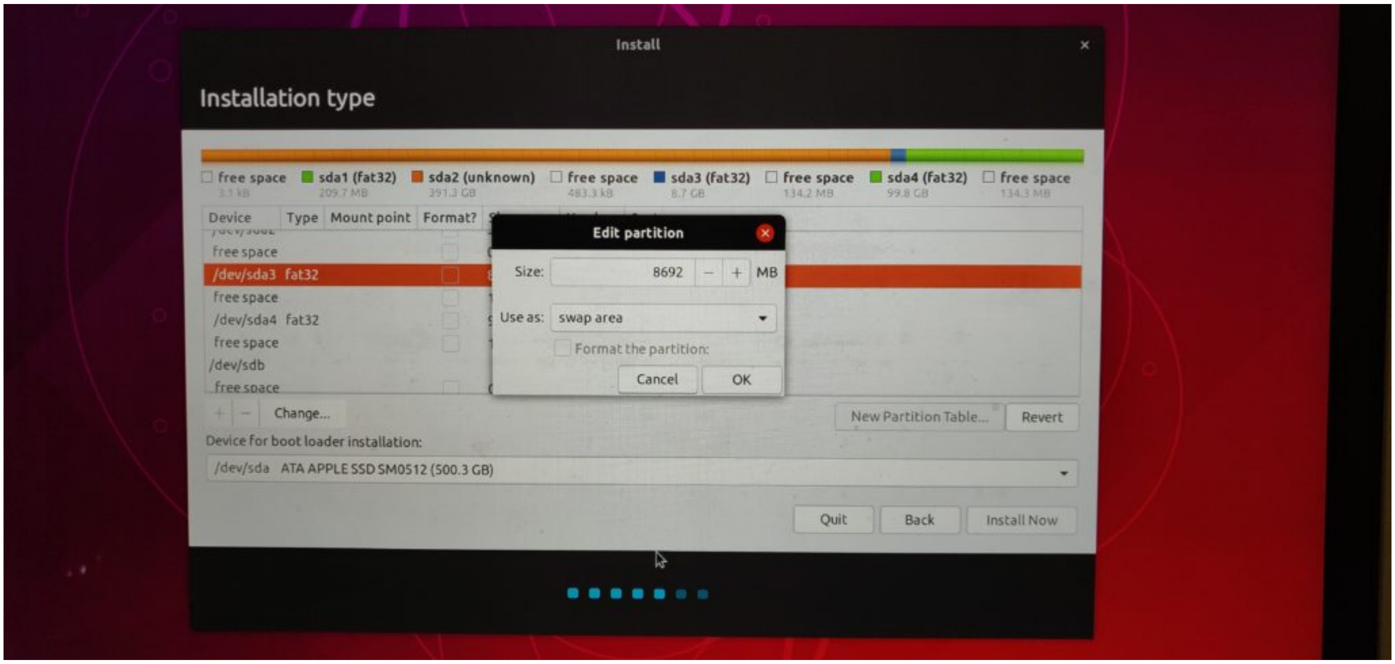


You can see the two fat32 partitions created with the Apple Disk Utility. In my case sda3 (8.7GB) and sda4 (99.8 GB).

You'll probably see three FAT32 partitions. One of them will be near the start of the disk and won't resemble the capacity of the partitions you created. This is the EFI boot partition. It's tiny (209.7 MB). Make sure you leave this partition well and truly alone, otherwise you'll possibly not be able to boot your mac!

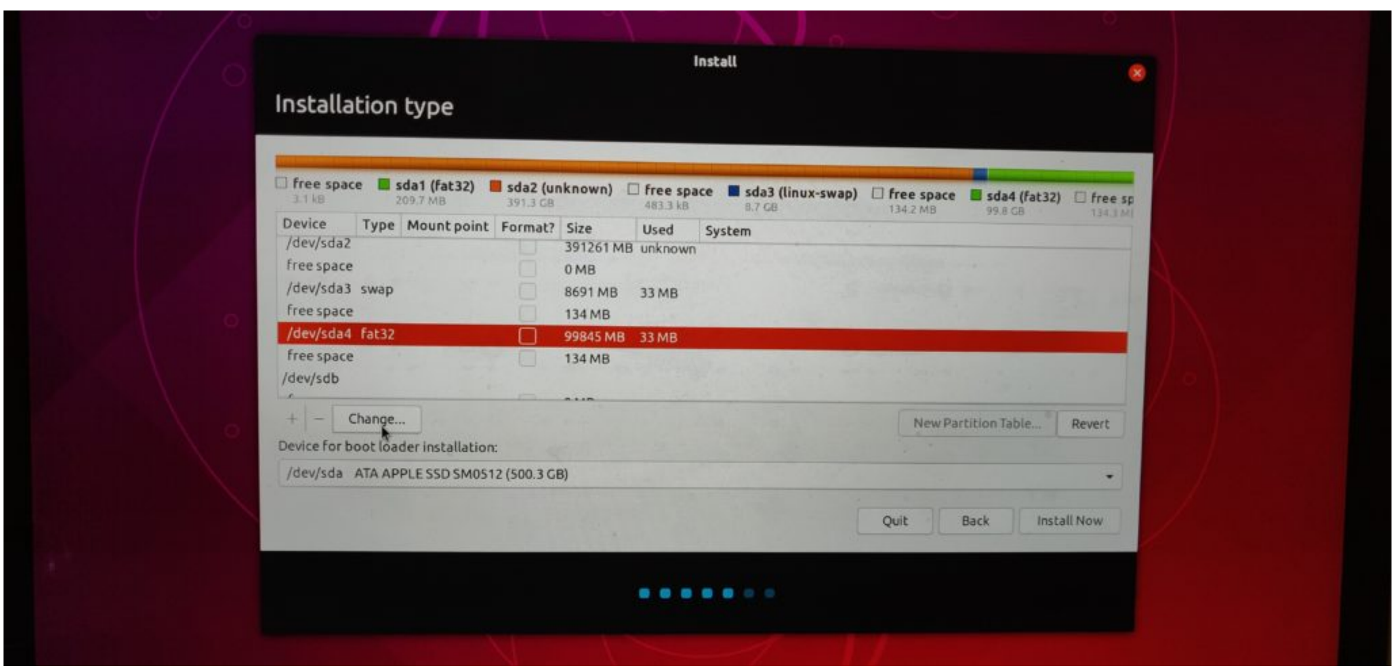
If you didn't create a swap partition, don't worry, you can still do so by locating the empty partition you made and create 2 partitions out of it. Simply make a big partition and a small partition (roughly 8-16 GB in size). The big partition should be the remainder of the free space. The big partition should be ext4 in type, and should be formatted with the mount point of "/". The small partition should be formatted as swap.

It's time to set up the partitions to use Linux. To do that, I selected my first (smaller) partition, the one that's 8.7GB. I'm going to use that as the Swap partition. Select that partition by clicking on the entry for it in the list of partitions. In my case, that's /dev/sda3. It must be of type fat32.

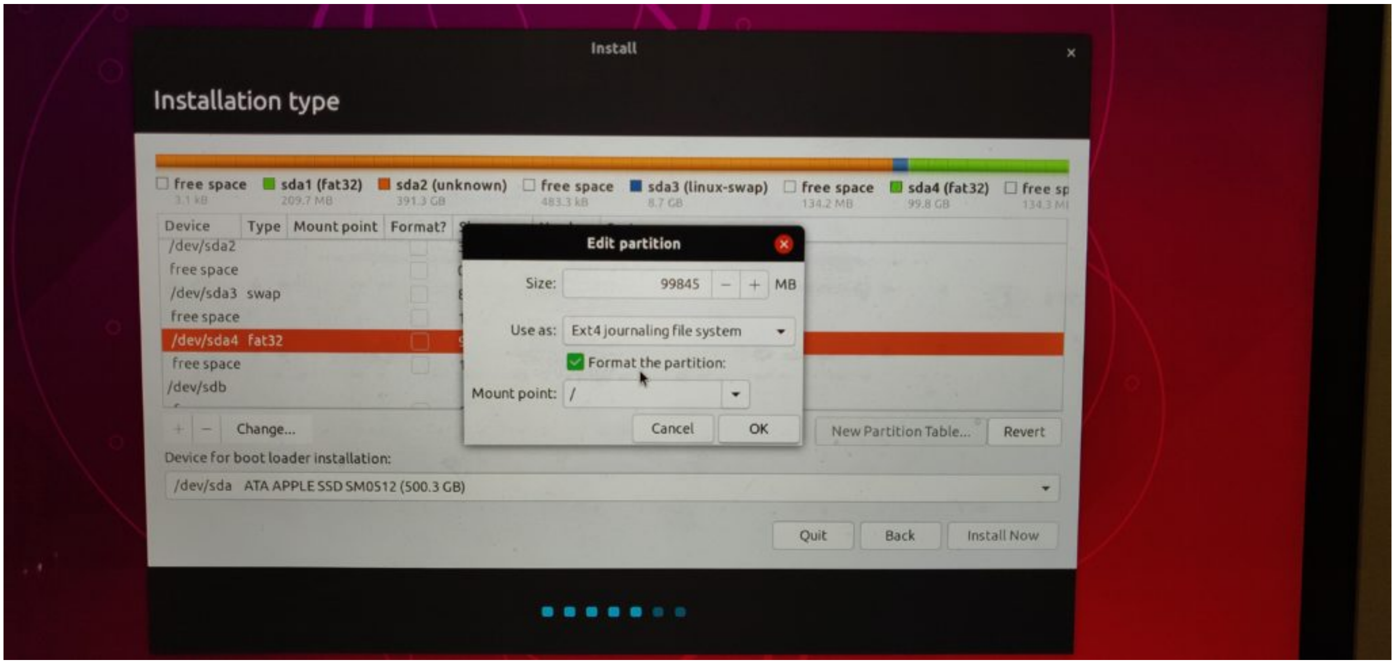


Once you click on it, click the button that says 'Change'. A dialogue saying 'Edit partition' will appear. Leave the size as it is, but click on the drop down which will probably say 'do not use'. Select 'swap area' from this list. Press OK.

Next, you want to allocate the large partition to be the main Linux partition (it's called /). Click on the large partition created in Disk Utility (in my case, /dev/sda4). It also has a type of fat32.

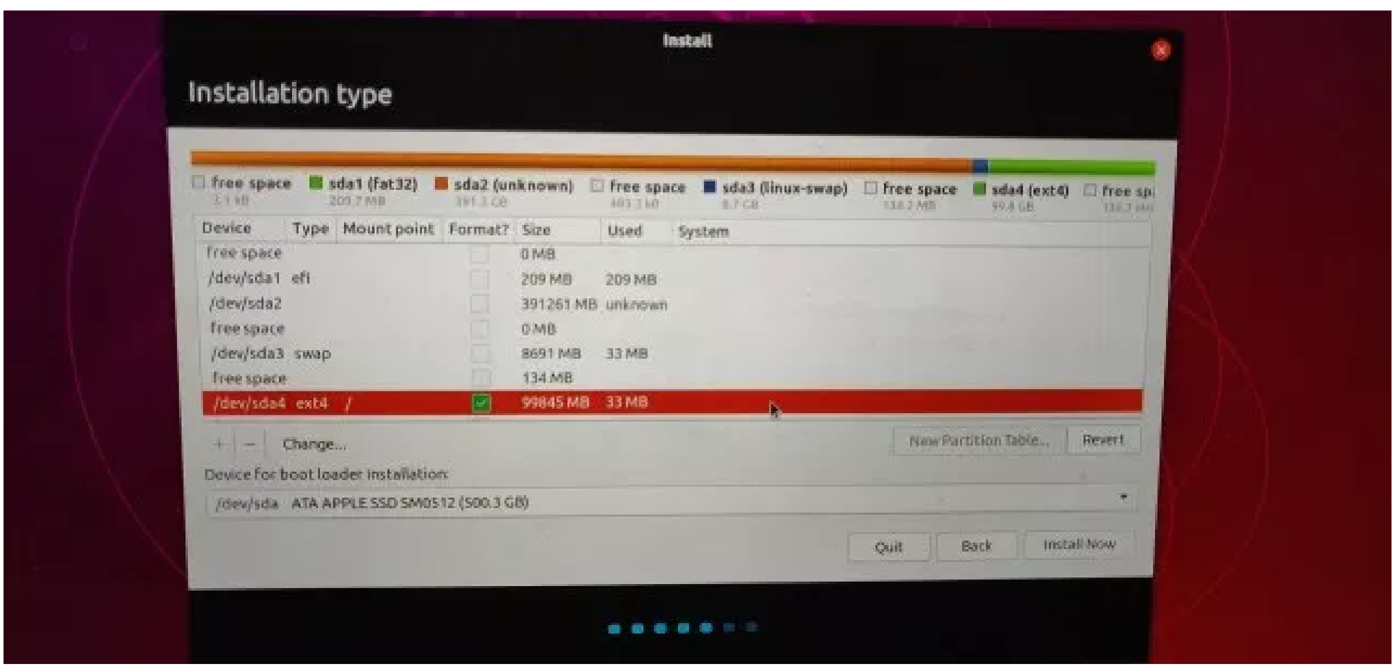


Clicking the 'Change' button will bring up the now familiar Edit Partition dialogue box. Again, leave the size as is, and from the 'Use as' drop-down, select ext4.



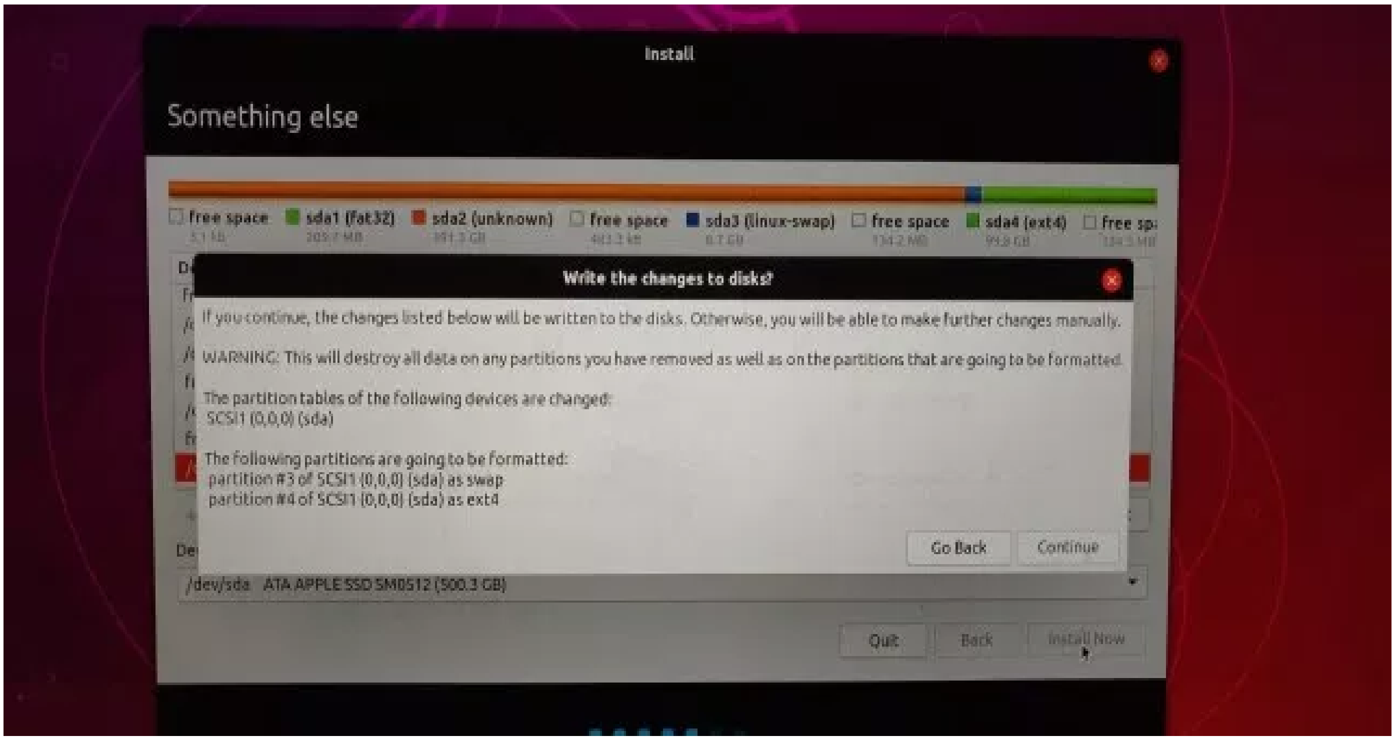
Click on 'Format this partition' if it isn't already ticked. By default, the mount point will be / - leave that as is. Click OK.

If you've done everything right, you'll now have two partitions. One which is small, of type swap and the other, the larger of the two, will be formatted as Linux ext4. These partitions will lie in amongst the other 'unknown' partitions (these are your macOS partitions).



This is what my setup looked like before pressing the Install Now button.

Once you are happy, click the Install Now button. You'll see a dialogue box asking you to confirm the changes are to be written to disk. This is your last chance before Ubuntu goes off and does it's thing to your disk. Again, I can't stress how important it is that you've taken that Time Machine backup with your mac before you do this. Anyway, I'm sure you've backed everything up... right? :) So click on 'Continue' and let the good times roll! Everything else should be pretty standard as per the normal Ubuntu installation.



Once the install has finished, the installer will tell you to remove the installation medium. Remove the USB stick and it enter to restart the computer. Once you hear the tell-tale Apple chime, hold down the alt/option key. Once again, you'll see your MacOS hard drive, as well as the newly installed Linux system. It'll probably be called 'EFI Boot'. Make sure you select that.

That's it! With any luck, Ubuntu should start up in a few moments and you're able to use your snazzy mac hardware with a better operating system! However, you may note that you probably won't have a few things that work out of the box. Most of these will be be covered off on the next step.

Step 4: Finishing up and fixing a few problems

CPU Running Hot?

If, like me, you notice that the mac starts running hot and the CPU fans are burning away then have a look at the output of the CPU history in the resources view of the System Monitor app (or using *top* at the terminal), you'll probably find that a 'kworker' process is chewing up CPU. This is a well known bug, so to fix this, run the following commands at the terminal:

```
$sudo -s
grep . -r /sys/firmware/acpi/interrupts/
```

You'll see a list of probably 70 or so lines relating to the firmware that works with ACPI (Advanced Configuration and Power Interface). Most of these are doing their thing quite happily, but you'll find one (or maybe even two) of them that has a number like gpe*16* has a large number beside it. It'll look like this:

```
/sys/firmware/acpi/interrupts/gpe16:  225420     STS enabled      unmasked
```

When you think you've found it, you can simply disable it, but first, just back up the file, just in case you make the wrong change. Note I am using gpe16 as that's the one I found the problem with, yours is probably different:

```
cp /sys/firmware/acpi/interrupts/gpe16 /root/gpe16.backup
echo "disable" > /sys/firmware/acpi/interrupts/gpe16
```

If after a few seconds (say 30-60), the CPU fans stop whirring, and system monitor/top starts showing normal usage statistics, then you know it's the right one. If it isn't the right one simply echo "enable", rather than disable.

To make the change permanent, do the following tasks, again at the terminal, changing the value '16' to the value you used:

```
# crontab -e

  --Add the below line to the crontab, so it will be executed every
startup/reboot:

@reboot echo "disable" > /sys/firmware/acpi/interrupts/gpe16

  -- Save/exit. Then, to make it work also after wakeup from suspend:

# touch /etc/pm/sleep.d/30_disable_gpe16
# chmod +x /etc/pm/sleep.d/30_disable_gpe16
# vim /etc/pm/sleep.d/30_disable_gpe16

  -- Add this stuff:

#!/bin/bash
case "$1" in
    thawresume)
        echo disable > /sys/firmware/acpi/interrupts/gpe16 2>/dev/null
        ;;
    *)
        ;;
esac
exit $?
```

Accessing your Macintosh files from Linux

Okay cokey. Now here's the thing. Apple can be real pains in the asses some times (read, all the time, at least these days). It is quite likely that you have what's called CoreStorage, if you have anything OS X 10.10 or newer. This provides an encrypted, journaled file system; even if you haven't installed FileVault (if you have, turn that off!).

To give full read/write access to your Mac OS X partition from Linux, you will need to revert it back to standard HFS+. To do this, you can pretty much enter one simple non-destructive command.

First up, at the terminal, issue the command **diskutil cs list**. You will see something like the below. If you know LVM in Linux, this is pretty much the same thing. Your main Mac OS X partition (Logical Volume) should be in Apple_HFS format.

```
CoreStorage logical volume groups (1 found)
|
+-- Logical Volume Group 56EDEE99-57E6-495B-A809-7042CBB9F725
    =========================================================
    Name:         Yosemite
    Status:       Online
    Size:         39349997568 B (39.3 GB)
    Free Space:   259604480 B (259.6 MB)
    |
    +-< Physical Volume 3D2DC4ED-4F33-4808-92F7-BCD6BFC36CA2
    |   ----------------------------------------------------
    |   Index:    0
    |   Disk:     disk0s4
    |   Status:   Online
    |   Size:     39349997568 B (39.3 GB)
    |
    +-> Logical Volume Family 993EBF39-FE2E-4F2C-BD44-76E460B9EC7A
        ----------------------------------------------------------
        Encryption Status:       Unlocked
        Encryption Type:         None
        Conversion Status:       NoConversion
        Conversion Direction:    -none-
        Has Encrypted Extents:   No
        Fully Secure:            No
        Passphrase Required:     No
        |
        +-> Logical Volume 47F9D6B1-F8F2-4E64-8AD4-9F2E2BD78E29
            ---------------------------------------------------
            Disk:                  disk2
            Status:                Online
            Size (Total):          38754844672 B (38.8 GB)
            Conversion Progress:   -none-
            Revertible:            Yes (no decryption required)
            LV Name:               Yosemite
            Volume Name:           Yosemite
            Content Hint:          Apple_HFS
```

As long as the 'Revertible' flag is set to Yes, you are good to go. Simply enter the following command:
`diskutil coreStorage revert`

The long string of stuff is that big long alphanumeric string of text highlighted in the red box, you want to use copy and paste it to make sure you don't make a mistake!

The conversion took ages for me, however your mileage may vary, depending upon how much data is on your drive, and how fast your drive is. If you type diskutil cs list again, you'll see how much % of the conversion has been accomplished. Don't reboot your machine until that's over and done with, but after then, you can safely mount your OS X partition with full read/write access.

First, make sure that you have hfsprogs installed. Example installation command:

```
sudo apt-get install hfsprogs
```

Next, mount or remount the HFS+ drive; commands need to be as follows:

```
sudo mount -t hfsplus -o force,rw /dev/sdXY /media/mntpoint
```

or

```
sudo mount -t hfsplus -o remount,force,rw /dev/sdXY /mount/point
```

If you want it to mount each time you start up your tux-ified Macintosh, you'll need to add the entry to the fstab (sudo vi /etc/fstab):

```
/dev/sdXY /media/mntpoint hfsplus force,rw,gid=1000,umask=0002 0 0
```

Where your user gid is 1000 (use the id command to find out your gid)

FaceTime HD Camera:

You'll need the FaceTime HD module for your kernel. It's a bit of a pain in the butt to get going, but it does go once you've set it up. Full documentation is here: https://github.com/patjak/bcwc_pcie/wiki/Get-Started#get-started-on-ubuntu

Here are the steps I followed to get everything working on Ubuntu. You need to be running a fairly recent version of Ubuntu (16.04 onwards should be fine), so 18.10 will be no worries. You'll need to run all the following commands from the Terminal.
`$` indicates running the command as a normal user

`$` indicates running the command as a normal user
`#` indicates as root (use the sudo command), eg: $sudo apt-get install …

- Install the dependencies : `# apt-get install linux-headers-`uname -r` git kmod libssl-dev checkinstall curl xzcat cpio`
    - (Note that xzcat is called xz-utils on Ubuntu 18.10).
- Extract and install the firmware file:
    - `$ git clone https://github.com/patjak/bcwc_pcie.git`
    - `$ cd bcwc_pcie/firmware`
    - `make`
    - `sudo make install`
- The output should say '`Copying firmware into '/usr/lib/firmware/facetimehd'`
- Now you need to build the kernel module (driver). Change into that dir: `$ cd ..`
- (you should now be in the bcwc_pcie folder)
- Build the kernel module: `$ make`
- Generate dkpg and install the kernel module, this is easy to uninstall later: `# checkinstall`
  Run depmod for the kernel to be able to find and load it: `# depmod`
- Load kernel module: `# modprobe facetimehd`
- Try it out by installing like 'cheese' and seeing if your webcam works.

No video device, or /dev/video does not exist?

I had a problem with the driver at this point, where /dev/video was not there, which was easily fixed by performing the following steps:

In some scenarios, you'll have to unload `bdc_pci` before inserting the kernel module, or `/dev/video` (or `/dev/video0`) won't be created. Do this with `modprobe -r bdc_pci`. If you've already done a modprobe facetimehd, also do a modprobe -r facetimehd, before re-running modprobe facetimehd. This fixed the issue for me.

Making the camera work on startup

If you want the driver to be enabled on startup, extra steps may be required. On Ubuntu, the following should work:

```
$sudo echo facetimehd >> /etc/modules
```

`sudo gedit /lib/systemd/system-sleep/99facetimehd` or if /lib/systemd/system-sleep does not exist: `sudo gedit /usr/lib/systemd/system-sleep/99facetimehd`

Paste this in the empty file:

```
#!/bin/sh
case $1/$2 in
  pre/*)
    echo "Going to $2..."
    modprobe -r facetimehd
    ;;
  post/*)
    echo "Waking up from $2..."
    modprobe -r bdc_pci
    modprobe facetimehd
    ;;
esac
```

And save.

Make it executable: `sudo chmod a+x /lib/systemd/system-sleep/99facetimehd` or `sudo chmod a+x /usr/lib/systemd/system-sleep/99facetimehd`

Making sure when you update your system your facetimehd driver updates too

When you perform a system update in Ubuntu, it often updates the Kernel too. When you update the kernel, the modules need to be upgraded to work with that Kernel version. As you've build a custom module, you'll need to ensure that the module is up to date too. Here's how to do that:

You will need to verify `dkms.conf` that the module name `facetimehd` and version number `0.1` are correct and either update the `dkms.conf` or adjust the instructions where `–m` and `–v` are used.

- Install needed packages: `# apt install debhelper dkms`
- Remove old package if installed: `# dpkg -r bcwc-pcie`
- Make a directory to work from: `# mkdir /usr/src/facetimehd-0.1`
- Change into the git repo dir: `$ cd bcwc_pcie`
- Copy files over: `# cp -r * /usr/src/facetimehd-0.1/`
- Change into that dir: `# cd /usr/src/facetimehd-0.1/`
- Remove any previous debs and backups: `# rm backup-*tgz bcwc-pcie_*deb`
- Clear out previous compile: `# make clean`
- Register the new module with DKMS: `# dkms add -m facetimehd -v 0.1`
- Build the module: `# dkms build -m facetimehd -v 0.1`
- Build a Debian source package: `# dkms mkdsc -m facetimehd -v 0.1 --source-only`
- Build a Debian binary package: `# dkms mkdeb -m facetimehd -v 0.1 --source-only`
- Copy deb locally: `# cp /var/lib/dkms/facetimehd/0.1/deb/facetimehd-dkms_0.1_all.deb /root/`
- Get rid of the local build files: `# rm -r /var/lib/dkms/facetimehd/`
- Install the new deb package: `# dpkg -i /root/facetimehd-dkms_0.1_all.deb`

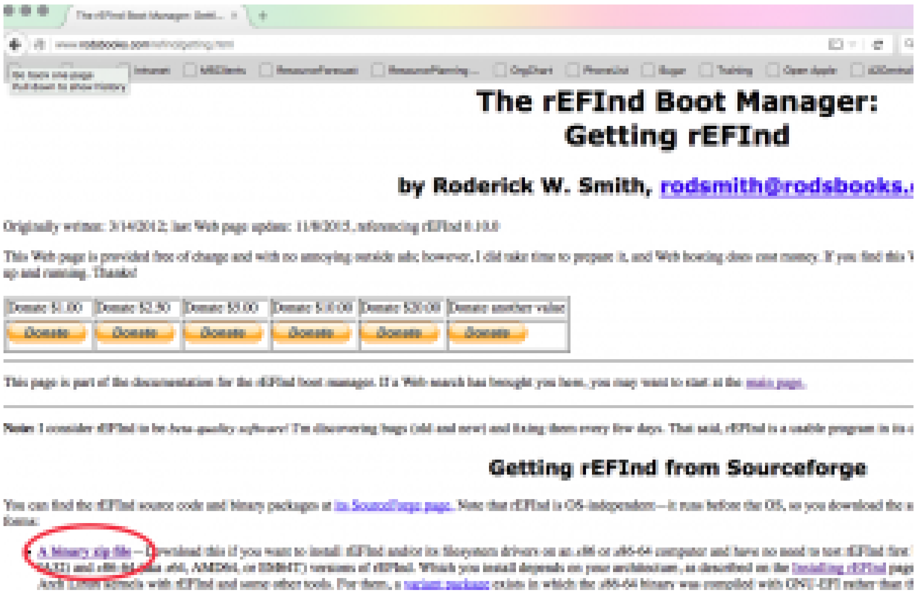
If you have any trouble, please read this guide on making a DKMS package:http://www.xkyle.com/building-linux-packages-for-kernel-drivers/

Problems booting Linux? Fix it by installing the EFI boot manager and disabling SIP protection.

Hopefully the following section won't bug most of you any more. With recent versions of most Linux distros supporting EFI, this shouldn't be an issue. However, if you are having issues getting your distro to boot, then read on.

EFI stands for Extensible Firmware Interface and is now pretty much commonplace in Macs and PCs across the industry. It replaced the trusty old BIOS system that PCs had used since the 1980s. Installing Linux on a BIOS based machine was trivial, but now with Apple's take on EFI on their customised hardware, it can be a little challenging. No worries, this is the Ultimate Linux Newbie Guide. We got this!

Download rEFInd



The red circle indicates where to download rEFInd

rEFInd is a boot-loader for EFI based machines. Think of it like bootcamp, or GRUB for GRUB :) You'll want to download rEFInd from the rEFInd website:

- **rEFInd Website**

Now, if you take a look around the rEFInd website, you'll see it looks like the guy that wrote it believes in punishing everyone that wants to use it. It took us about 20 minutes just to find the frigging download link! So the ULNG has taken the time to go through all the pertinent steps to make it shit tons easier for you!

The version of rEFInd that we used is 0.10.0, and we used the zip archive version. Once you download the binary, you are going to need to start the rest of your work from the Terminal, so open up the Terminal from the Utilities folder on your Macintosh and head over to your Downloads folder where you saved rEFInd to.

If the zip archive is not already unzipped, unzip it using the unzip command and head into the newly created refind-bin-0.11.0 folder:

```
$unzip refind-bin-0.11.0.zip

$cd refind-bin-0.11.0
```

For the next step, take a note of the full directory where you downloaded the refind tool into. For example /Users/bob/Downloads/refind-bin-0.11.0 (you can also type pwd at the command prompt to tell you which present working directory you are in).

Installing rEFInd by working around SIP

Before we can properly install rEFInd, we will need to take care of a pesky thing that Apple put into their hardware called SIP (System Integrity Protection). There are a couple of ways to do this, but I found the easiest way to do so is to pop your system into recovery mode and issue a command from the terminal there. There is a bit more information on this process over here.

To enter recovery mode on your Macintosh, shut your machine down completely. Give the machine around 30 seconds and then switch back on. Now quickly hold down the Command and R key at the same time until at least you hear the Apple 'chime' sound. Shortly you will enter recovery mode. I recommend plugging in an Ethernet cable to do this, however it is possible to do with WiFi.

Once you are in the Recovery tool, enter the Utilities menu up on the top bar, and click on Terminal.

Issue the following command:

```
csrutil disable
```

NOTE: Using macOS from Sierra onwards, the csrutil tool may have been removed. If csrutil is unavailable for whatever reason, don't despair, simply go into the directory that you downloaded refind into and run refind-install. Earlier, you noted down this folder, so just cd to it, for example:

```
$cd /Users/bob/Downloads/refind-bin-0.11.0/
```

Once you have done that, install rEFInd:

```
sudo ./refind-install
```

(if you are prompted for a password, note that this is your own mac password).

NB: if you have issues and find that rEFInd doesn't operate properly, you can also try the --alldrivers flag (but use this with extreme caution!) `$sudo ./refind-install --alldrivers`

Once REFind is all installed, reboot the mac and you should be good to go. All going well, you should be seeing the rEFInd menu. Use the cursor key to select your Linux installation and hit that return key. Fingers crossed, your system will start up without much of a hitch!

If you don't see the rEFInd menu on startup, try starting up your mac whilst holding down the Command key (or if that doesn't work, the alt/option key).

---YOU PROBABLY NO LONGER NEED THE BELOW INFORMATION!---

The next bit of text was necessary for versions of rEFInd before 0.10.0. This guide has been updated for version 0.11.0, and so you shouldn't need to do any of this. Isn't that great?! However, if things don't work the way you expect, then you can do this whilst still in the recovery tool, and in the refind folder.

Now it's time to edit the EFI config file, but you will need to mount that hidden EFI partition first. Thankfully, rEFInd has a little tool you can use to mount the partition:

```
$sudo mountesp
```

Edit /Volumes/ESP/EFI/refind/refind.conf. Like me, you may find the refind.conf file is in /Volumes/ESP/EFI/BOOT, instead of a folder called refind.

```
$sudo nano /Volumes/ESP/EFI/refind/refind.conf (or use vi like
me, if you are that way inclined. Just not emacs!).
```

locate the line that says scanfor and edit it to say:

```
scanfor internal
```

If no such line exists, add it into the file near the top.

Next, change the config file to load the appropriate Linux file system driver. Check for a line that starts fs0. If no such line exists, add it as below, otherwise edit it:

```
fs0: load ext4_x64.efi
```

```
fs0: map -r
```

Save the file and quit your editor. That's pretty much it for the rEFInd bit. That is the hardest part over and done with. If you want to be sure it worked, you should power off your machine and power on again. If you see a grey screen with the rEFInd logo, then it has worked. You should be able to chose the Mac OS X logo and hit return to start up OS X again.

Screen backlight, Keyboard Backlight and Volume control hotkeys

I haven't had any issues with the screen backlight, keyboard backlight and the volume control keys since Ubuntu 17.10, however if you do, a package is now available for Debian and Ubuntu called 'pommed', which handles the hotkeys found on the Apple MacBook Pro, MacBook and PowerBook laptops and adjusts the LCD backlight, sound volume, keyboard backlight or ejects the CD-ROM drive accordingly.

Installation is as simple as installing the package through apt-get:

```
sudo apt-get install pommed
sudo pommed
```

This will run pommed as a daemon (run in the background).

If that doesn't work for whatever reason run it in the foreground and check for any errors `sudo pommed -f`. On my Late 2013 Macbook Pro Retina 15", pommed did not work for me. Check out Jessie's blog and accompanying script for a more manual solution if you face this problem too.

NB: I did find that my keyboard backlight buttons now work out of the box on Ubuntu 17.10.

Nvidia Graphics & Retina Display

The graphics display should generally work out of the box, however there may be 'interesting' graphical issues. Not all of these might be fixable, but give the NVidia drivers a try, and if you still don't have any luck, read the many forums until you get a solution that works for you.

```
sudo apt-get install nvidia-driver xserver-xorg-video-intel
```

Note if you are not using xorg, you'll need to make the appropriate changes here. Maybe best to stick with xorg for now!

On newer macs, they use AMD graphics rather than NVidia. They also have their own set of unique problems in some cases. As I don't have a mac with AMD graphics, you'll need to do a little more googling on that.

Your Macbook Pro Retina display is also known outside the Apple world as an HiDPI display (high resolution graphics). Using the nvidia driver ensures that the maximum resolution of your display is achieved, however if you are used to seeing things extra small (therefore more screen real-estate, you can enable HiDPI scaling for GNOME via the following Terminal command and log out and log back into GNOME:

```
gsettings set org.gnome.desktop.interface scaling-factor 1
```

Setting it to a value of 2 returns the display to how it was before. You can also edit this setting within the dconf editor (GUI application)

If you are using another window manager such as KDE or are having issues with other apps not playing nicely, have a look at the ArchWiki for hints on HiDPI.

Okay, that about wraps it up for this ditty, I hope it has worked for you. If it hasn't, or you have some feedback to offer, we would love to hear it!